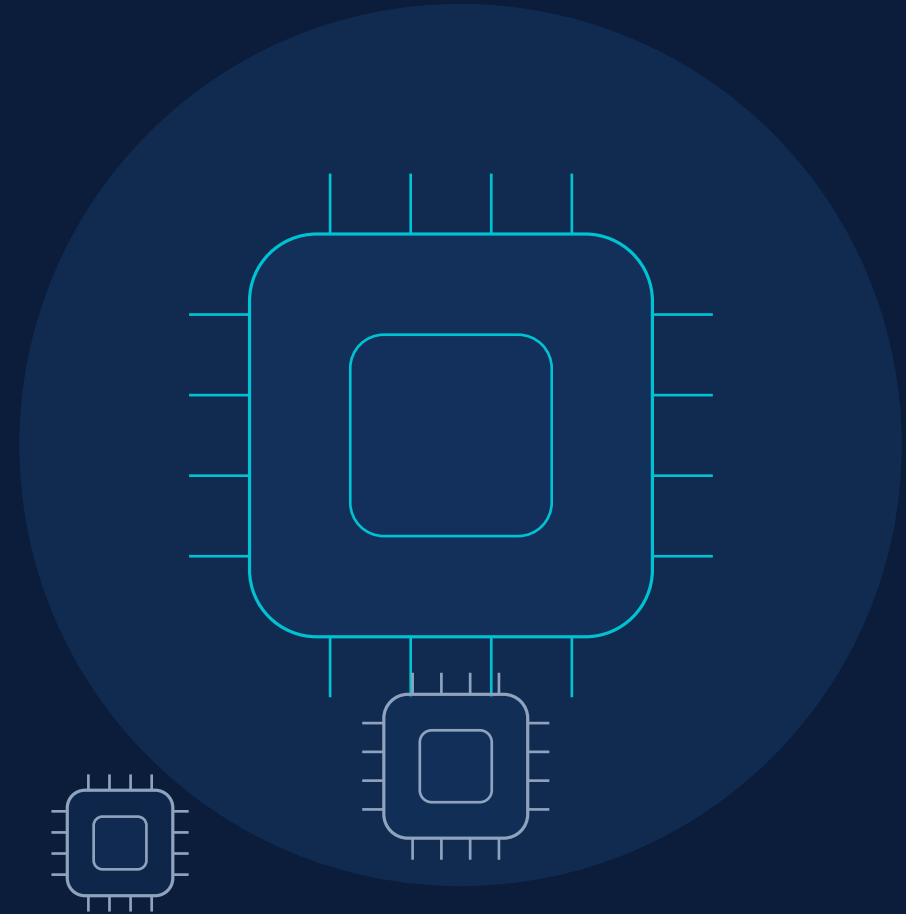


MARCO 3

Classificador de Dígitos Manuscritos (MNIST)

Inferência ELM em SoC Heterogêneo ARM + FPGA

Aplicação C · Validação · Benchmark · Métricas



Visão Geral do Sistema



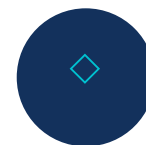
Comunicação via Lightweight HPS-to-FPGA Bridge (MMIO) e drivers Linux em Assembly ARM

Objetivo do Marco 3



Aplicação C completa

CLI com classificação individual, leitura de dataset e modo de benchmark



IP-Core VGA integrado

Exibição da imagem (arquivo ou desenhada) na tela durante a inferência



Demonstração final

Execução de dataset de teste com apresentação de métricas



Documentação completa

Arquitetura final, resultados de acurácia/desempenho e análise de gargalos

Arquitetura da Aplicação C



Inferência a partir de Arquivo



Caminho do arquivo recebido via linha de comando, conforme requisito da aplicação.

1

Carregar imagem

Lê PNG 28x28 em escala de cinza (stb_image) ou .bin bruto de 784 bytes

2

Exibir no VGA

Upscale do quadro 28x28 para 240x240 centralizado em 320x240, via Pixel Buffer Controller

3

Enviar ao coprocessador

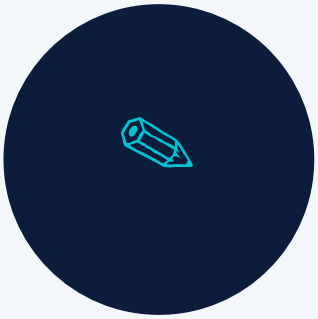
carregar_e_inferir() reseta o núcleo, transmite a imagem e aciona a inferência via MMIO

4

Exibir resultado

Dígito previsto (0–9) impresso na interface em modo texto

Inferência a partir de Desenho



Mesmo pipeline de inferência do modo Arquivo, alimentado pelo buffer desenhado.

1

Captura de mouse

Leitura direta de `/dev/input/mice` e teclado em modo raw via termios

2

Grade 28x28 sobre o VGA

Cursor mapeado da resolução de tela para a grade lógica; clique esquerdo pinta, direito apaga

3

Suavização (anti-aliasing)

Kernel gaussiano 3x3 aplicado às células vizinhas para aproximar o traço ao estilo MNIST

4

Comandos

Enter = inferir · C = limpar tela · P = salvar PNG · Q = sair do modo

Validação e Benchmark



1

Amostragem balanceada

Seleciona aleatoriamente N/10 imagens de cada subpasta de dígito (0 a 9) do dataset

2

Execução em lote

Cada imagem passa pelo mesmo pipeline de inferência, com medição individual de latência

3

Métricas calculadas

Acurácia (%), latência média e desvio padrão (ms), throughput (imagens/s)

4

Logs da execução

historico_inferencias.csv registra cada inferência; historico_usuario.csv registra eventos do menu, erros e conclusão.
benchmark.csv segue ignorado.

Métricas do Modo Benchmark



XX,X %

Acurácia



XX,X ± X,X ms

Latência média ± desvio



XX,X img/s

Throughput

Conferir métricas no terminal e no historico_inferencias.csv. Se a placa gravar 1970, ajuste antes do teste: sudo date -s "AAAA-MM-DD HH:MM:SS"

Antes do benchmark: confira a data da placa para o timestamp do log não sair como 1970.

Pipeline de Inferência (carregar_e_inferir)



Logs separados: registrar_log_csv() acompanha inferências; log_evento() acompanha ações do usuário e erros de entrada.

Desafios Técnicos e Soluções



Protocolo de pixel no VGA

Pacote de 32 bits (x, y, RGB de 3 bits e strobe) escrito no registrador do Pixel Buffer Controller.



Escalonamento 28×28 → 240×240

Reamostragem por vizinho mais próximo com centralização em tela 320×240.



Leitura de mouse em baixo nível

Decodificação de pacotes PS/2 de /dev/input/mice, incluindo extensão de sinal dos deltas.

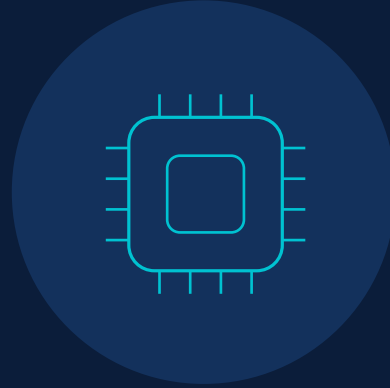


Anti-aliasing dos traços

Convolução com kernel gaussiano 3×3 e realce de intensidade para aproximar o desenho ao MNIST.

Requisitos do Marco 3 Atendidos

- ✓ Aplicação C com CLI: classificação individual, dataset e benchmark
- ✓ Integração com IP-Core VGA para exibição da imagem inferida
- ✓ Desenho via mouse com mapeamento para grade 28×28
- ✓ Recebimento de caminho e parâmetros via linha de comando
- ✓ Envio da imagem ao coprocessador e impressão do dígito previsto
- ✓ Cálculo de acurácia, latência média/desvio e throughput
- ✓ Logs separados: historico_usuario.csv para eventos/erros e historico_inferencias.csv para predição, resultado, latência e status



Obrigado!

Perguntas e discussão